Code and branch coverage analysis

L6_L7_53_bike_share_rental / Python application: bike_share_rental.py

Student: Bhavesh V **SRN:** PES1UG24CS115 **Section:** B

Final result

Measure	Recorded result	Target
Line coverage	40 / 40 executable statements = 100%	At least 90%
Branch coverage	30 / 30 measured outcomes = 100%	At least 90%
Final test suite	57 passed; 0 failed	All assertions pass
Missing / partial	0 missing statements; 0 partial branches	No uncovered measured outcomes

The original application is unchanged. Coverage is restricted to the application module, so executing test code does not inflate the reported percentages. No source lines or branches are excluded.

Initial measurement and improvement

The two starter tests passed but executed only 20 of 40 statements (**50.00% line coverage**) and 11 of 30 branch outcomes (**36.67% branch coverage**). The initial branch-mode terminal report displayed **44%**: its Cover column combines statements and branches, (20 + 11) / (40 + 30) = 44.29%, rounded for display.

Stage 1 adds a maintained suite of 17 cases and reaches 100% line coverage. Stage 2 retains those cases and adds 40 decision and boundary cases. The final standalone branch suite passes 57 cases and reaches 100% on both measures.

Coverage concepts

Line coverage measures executed statements. **Branch coverage** measures alternative control-flow destinations. **Function coverage** asks whether each function was invoked; all three application functions were invoked. **Path coverage** considers complete routes through the program and is not established by these reports. Full measured coverage does not prove correctness.

Test design and coverage reasoning

Tests use fixed expected prices, status strings, exception messages and boolean results. Parametrization keeps individual input combinations independently visible to pytest; the final suite contains 57 collected cases.

Area	Inputs and asserted behavior
Ride prices	All three rates; 1, 29, 30, 31, 60 and 61 minutes test block rounding. A 30-minute standard ride costs 20; 31 minutes costs 40.
Subscription and peak	First block is free. A subscriber ride up to 30 minutes returns 0 even at peak. Cargo is exempt from peak surcharge; standard adds 15.
Long rides	180 minutes does not add the long-ride fee; 181 does. Standard non-subscriber prices are 120 and 240. An electric subscriber peak ride at 181 minutes costs 6 x 35 + 15 + 100 = 325.
Validation	Unknown bike; zero/negative minutes; zero/negative capacity; negative bike count and bike count above capacity each raise ValueError.
Dock decisions	All five statuses; 0/full and 1/2/3 bike boundaries; 1/2/3 free-slot boundaries. Empty/full take precedence at capacity 1. With 1 bike and capacity 3, few bikes takes precedence over few slots.
Unlock decisions	Active rides deny unlocking. Ages 15, 16, 17 and 18; teen balances 99/100 and adult balances 49/50. Both passing and failing balance comparisons are asserted, including a negative adult balance.

Measured branches by function

Function	Statements covered	Branch outcomes covered
ride_cost	15 / 15	12 / 12
dock_status	14 / 14	12 / 12
can_unlock	7 / 7	6 / 6
Module-level statements	4 / 4	0 (no decisions)

Coverage.py measures line-to-line branches. Boolean return comparisons and short-circuit operands may not produce separate measured arcs. The explicit threshold and compound-condition cases strengthen behavior checks beyond the coverage total; this is not a claim of exhaustive condition or path coverage.

Execution, evidence and repository history

Environment and reproduction

Executed on 03 November 2026 (Asia/Kolkata): Python 3.12.13, pytest 9.2.0, coverage.py 7.17.0. Test dependencies are pinned in LAB6/requirements.txt.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r LAB6/requirements.txt
python LAB6/run_coverage.py line
python LAB6/run_coverage.py branch
python -m pytest -q
```

Run from the repository root. The runner switches to LAB6, runs the named test file explicitly and uses independent COVERAGE_FILE databases. It saves stdout, JSON, and HTML reports. The final ordinary pytest command also collected and passed all 57 cases.

Two implementation commits completed

Commit	Change
856e691	Complete line coverage tests with 100 percent coverage and evidence
9789f30	Complete branch coverage with 57 passing cases and 100 percent coverage

Both are local repository commits. The line commit includes the maintained line test file and screenshot; the branch commit includes the maintained branch test file and screenshot. Existing application code is preserved.

Evidence included in this single PDF

The next two landscape pages contain the genuine Terminal screenshots provided by the student after running the line and branch coverage commands. Each page preserves the complete supplied screenshot and includes an enlarged detail of its results table for readability. Only final results above 90% are shown as screenshots.

The appendices reproduce the complete application, both named test files and the default pytest entry point. Embedded PDF attachments preserve the runnable Python files, requirements, README, original Terminal screenshots, saved runner stdout, JSON coverage data and the two-commit history. A PDF reader with attachment support can extract them.

Source reference: Lab_6_Code&BranchCoverage_Student_Handout.docx (1).pdf, especially the steps and deliverables on pages 3-4. Repository evidence and local test executions supply all result values in this report.

Evidence 1 / Final line coverage

17 passed. All 40 executable statements covered; no missing lines. Genuine Terminal screenshot supplied by the student.

```
(.venv) bvell@BhaveshVellurus-MacBook-Air LAB6 % python -m coverage run --source=bike_share_rental -m pytest -q CodeCoverageTest_53_bike_share_rental.py
.................                                                                                                    [100%]
17 passed in 0.02s
(.venv) bvell@BhaveshVellurus-MacBook-Air LAB6 % python -m coverage report -m
Name                   Stmts   Miss  Cover   Missing
----------------------------------------------------
bike_share_rental.py      40      0   100%
----------------------------------------------------
TOTAL                     40      0   100%
(.venv) bvell@BhaveshVellurus-MacBook-Air LAB6 %
```

Enlarged coverage table from the same screenshot

```
Name                   Stmts   Miss  Cover   Missing
----------------------------------------------------
bike_share_rental.py      40      0   100%
----------------------------------------------------
TOTAL                     40      0   100%
```

Line coverage = 40 / 40 x 100 = 100.00%. The report records 40 statements, 0 missed statements and 100% coverage. The screenshot shows 17 passed in 0.02 seconds.

Evidence 2 / Final branch coverage

57 passed. 40/40 statements and 30/30 measured branch outcomes covered. Genuine Terminal screenshot supplied by the student.

```
(.venv) bvell@BhaveshVellurus-MacBook-Air LAB6 % python -m coverage run --branch --source=bike_share_rental -m pytest -q BranchCoverageTest_53_bike_share_rental.py
.....................................................                                    [100%]
57 passed in 0.04s
(.venv) bvell@BhaveshVellurus-MacBook-Air LAB6 % python -m coverage report -m
Name                   Stmts   Miss Branch BrPart  Cover   Missing
------------------------------------------------------------------
bike_share_rental.py      40      0     30      0   100%
------------------------------------------------------------------
TOTAL                     40      0     30      0   100%
(.venv) bvell@BhaveshVellurus-MacBook-Air LAB6 %
```

Enlarged coverage table from the same screenshot

```
Name                   Stmts   Miss Branch BrPart  Cover   Missing
------------------------------------------------------------------
bike_share_rental.py      40      0     30      0   100%
------------------------------------------------------------------
TOTAL                     40      0     30      0   100%
```

BrPart = 0 and Missing is empty. The explicit branch ratio is 30 / 30 = 100.00%; the combined Cover result is also 100%. The screenshot shows 57 passed in 0.04 seconds.

Appendix A / Application under test

Complete maintained source listing, lines 1-46. Numbers at left are report annotations, not part of the file.

```
1  RATE_PER_30_MIN = {"standard": 20, "electric": 35, "cargo": 50}
2
3
4  def ride_cost(bike_type, minutes, is_subscriber, is_peak):
5      if bike_type not in RATE_PER_30_MIN:
6          raise ValueError("unknown bike type")
7      if minutes <= 0:
8          raise ValueError("invalid duration")
9      blocks = (minutes + 29) // 30
10     if is_subscriber:
11         blocks -= 1
12         if blocks <= 0:
13             return 0
14     cost = blocks * RATE_PER_30_MIN[bike_type]
15     if is_peak and bike_type != "cargo":
16         cost += 15
17     if minutes > 180:
18         cost += 100
19     return cost
20
21
22 def dock_status(bikes_docked, capacity):
23     if capacity <= 0:
24         raise ValueError("invalid capacity")
25     if bikes_docked < 0 or bikes_docked > capacity:
26         raise ValueError("invalid bike count")
27     free_slots = capacity - bikes_docked
28     if bikes_docked == 0:
29         return "empty"
30     if free_slots == 0:
31         return "full"
32     if bikes_docked <= 2:
33         return "few bikes"
34     elif free_slots <= 2:
35         return "few slots"
36     return "available"
37
38
39 def can_unlock(account_balance, has_active_ride, age):
40     if has_active_ride:
41         return False
42     if age < 16:
43         return False
44     elif age < 18:
45         return account_balance >= 100
46     return account_balance >= 50
```

Application source is byte-for-byte identical to the original repository version (commit 17b7c82). SHA-256: 78526f0452e152b9e06de2435f215da19a51f7d7602309060ed1f1ee8c487c56

Appendix B / Line coverage tests

Complete maintained source listing, lines 1-69. Numbers at left are report annotations, not part of the file.

```
  1  """Stage 1: exercise every executable application statement."""
  2
  3  import pytest
  4
  5  from bike_share_rental import can_unlock, dock_status, ride_cost
  6
  7
  8  @pytest.mark.parametrize(
  9      "bike, minutes, subscriber, peak, expected",
 10      [
 11          ("standard", 45, False, False, 40),
 12          ("standard", 30, True, False, 0),
 13          ("electric", 60, True, True, 50),
 14          ("cargo", 181, False, False, 450),
 15      ],
 16  )
 17  def test_ride_cost_statements(bike, minutes, subscriber, peak, expected):
 18      assert ride_cost(bike, minutes, subscriber, peak) == expected
 19
 20
 21  @pytest.mark.parametrize(
 22      "bike, minutes, message",
 23      [
 24          ("tandem", 30, "unknown bike type"),
 25          ("standard", 0, "invalid duration"),
 26      ],
 27  )
 28  def test_ride_cost_errors(bike, minutes, message):
 29      with pytest.raises(ValueError, match=f"^{message}$"):
 30          ride_cost(bike, minutes, False, False)
 31
 32
 33  @pytest.mark.parametrize(
 34      "bikes, capacity, expected",
 35      [
 36          (0, 10, "empty"),
 37          (10, 10, "full"),
 38          (2, 10, "few bikes"),
 39          (8, 10, "few slots"),
 40          (5, 10, "available"),
 41      ],
 42  )
 43  def test_dock_status_statements(bikes, capacity, expected):
 44      assert dock_status(bikes, capacity) == expected
 45
 46
 47  @pytest.mark.parametrize(
 48      "bikes, capacity, message",
 49      [
 50          (0, 0, "invalid capacity"),
 51          (-1, 10, "invalid bike count"),
 52      ],
 53  )
 54  def test_dock_errors(bikes, capacity, message):
 55      with pytest.raises(ValueError, match=f"^{message}$"):
 56          dock_status(bikes, capacity)
 57
 58
 59  @pytest.mark.parametrize(
 60      "balance, active, age, expected",
 61      [
 62          (200, True, 25, False),
 63          (200, False, 15, False),
 64          (100, False, 16, True),
 65          (50, False, 18, True),
 66      ],
 67  )
 68  def test_unlock_statements(balance, active, age, expected):
 69      assert can_unlock(balance, active, age) is expected
```

Appendix C / Branch tests, part 1

Complete maintained source listing, lines 1-46. Numbers at left are report annotations, not part of the file.

```
1  """Stage 2: retain stage 1 and add decision and boundary assertions."""
2
3  import pytest
4
5  from bike_share_rental import can_unlock, dock_status, ride_cost
6  from CodeCoverageTest_53_bike_share_rental import (  # noqa: F401
7      test_dock_errors,
8      test_dock_status_statements,
9      test_ride_cost_errors,
10     test_ride_cost_statements,
11     test_unlock_statements,
12 )
13
14
15 @pytest.mark.parametrize(
16     "bike, minutes, subscriber, peak, expected",
17     [
18         ("standard", 1, False, False, 20),
19         ("standard", 29, False, False, 20),
20         ("standard", 30, False, False, 20),
21         ("standard", 31, False, False, 40),
22         ("standard", 60, False, False, 40),
23         ("standard", 61, False, False, 60),
24         ("electric", 30, False, False, 35),
25         ("cargo", 30, False, False, 50),
26         ("standard", 1, True, True, 0),
27         ("electric", 30, True, True, 0),
28         ("standard", 31, True, False, 20),
29         ("cargo", 60, True, True, 50),
30         ("standard", 30, False, True, 35),
31         ("cargo", 30, False, True, 50),
32         ("standard", 180, False, False, 120),
33         ("standard", 181, False, False, 240),
34         ("electric", 181, True, True, 325),
35     ],
36 )
37 def test_ride_boundaries(bike, minutes, subscriber, peak, expected):
38     assert ride_cost(bike, minutes, subscriber, peak) == expected
39
40
41 @pytest.mark.parametrize("minutes", [-1, -30])
42 def test_negative_duration(minutes):
43     with pytest.raises(ValueError, match="^invalid duration$"):
44         ride_cost("standard", minutes, False, False)
45
46
```

The five imported stage-1 test functions are collected with the new branch cases, allowing this file to run independently.

Appendix C / Branch tests, part 2

Complete maintained source listing, lines 47-88. Numbers at left are report annotations, not part of the file.

```
47  @pytest.mark.parametrize("bikes, capacity", [(0, -1), (11, 10)])
48  def test_remaining_dock_error_paths(bikes, capacity):
49      message = "invalid capacity" if capacity <= 0 else "invalid bike count"
50      with pytest.raises(ValueError, match=f"^{message}$"):
51          dock_status(bikes, capacity)
52
53
54  @pytest.mark.parametrize(
55      "bikes, capacity, expected",
56      [
57          (1, 10, "few bikes"),
58          (3, 10, "available"),
59          (7, 10, "available"),
60          (9, 10, "few slots"),
61          (0, 1, "empty"),
62          (1, 1, "full"),
63          (1, 3, "few bikes"),
64          (3, 4, "few slots"),
65      ],
66  )
67  def test_dock_boundaries_and_precedence(bikes, capacity, expected):
68      assert dock_status(bikes, capacity) == expected
69
70
71  @pytest.mark.parametrize(
72      "balance, active, age, expected",
73      [
74          (1000, True, 15, False),
75          (1000, True, 18, False),
76          (1000, False, 15, False),
77          (99, False, 16, False),
78          (100, False, 16, True),
79          (99, False, 17, False),
80          (100, False, 17, True),
81          (49, False, 18, False),
82          (50, False, 18, True),
83          (-1, False, 30, False),
84          (51, False, 30, True),
85      ],
86  )
87  def test_unlock_boundaries(balance, active, age, expected):
88      assert can_unlock(balance, active, age) is expected
```

Default pytest entry point: test_bike_share_rental.py

```
"""Default pytest entry point for the maintained Lab 6 test suite."""

from BranchCoverageTest_53_bike_share_rental import (  # noqa: F401
    test_dock_boundaries_and_precedence,
    test_dock_errors,
    test_dock_status_statements,
    test_negative_duration,
    test_remaining_dock_error_paths,
    test_ride_boundaries,
    test_ride_cost_errors,
    test_ride_cost_statements,
    test_unlock_boundaries,
    test_unlock_statements,
)
```

Use one entry point per run. The default filename supports normal pytest discovery; explicitly collecting it together with the named deliverables would collect the imported cases more than once.